

DSA Programs

Online Compiler Run Output

Programiz-style presentation • Code + Input + Output

Repository: [N.Greeshmanth-Varma-A25126510097-Dsa](#)



1. Binary Search – Employee ID

unit_1_a.java

```
1  import java.util.Scanner;
2  public class unit_1_a {
3      public static void main(String[] args) {
4          Scanner sc = new Scanner(System.in);
5          int[] a = new int[100];
6          System.out.print("Enter number of employee IDs: ");
7          int n = sc.nextInt();
8          System.out.println("Enter IDs in ascending order:");
9          for (int i = 0; i < n; i++) {
10             a[i] = sc.nextInt();
11         }
12         System.out.print("Enter ID to search: ");
13         int key = sc.nextInt();
14         int low = 0;
15         int high = n - 1;
16         int comparisons = 0;
17         boolean found = false;
18         while (low <= high) {
19             int mid = (low + high) / 2;
20             comparisons++;
21             if (a[mid] == key) {
22                 System.out.println("ID found at position " + (mid + 1));
23                 found = true;
24                 break;
25             } else if (key < a[mid]) {
26                 high = mid - 1;
27             } else {
28                 low = mid + 1;
29             }
30         }
31         if (!found) {
32             System.out.println("ID not found");
33         }
34         System.out.println("Number of comparisons = " + comparisons);
35     }
36 }
```

1. Binary Search – Employee ID — Run

Input used

```
5
101 105 110 115 120
115
```

Output

```
Enter number of employee IDs: 5
Enter IDs in ascending order:
101 105 110 115 120
Enter ID to search: 115
ID found at position 4
Number of comparisons = 2
```

Program finished successfully

2. Insertion Sort – Marks

unit_1_b.java

```
1  import java.util.Scanner;
2  public class unit_1_b {
3      public static void main(String[] args) {
4          Scanner sc = new Scanner(System.in);
5          System.out.print("Enter number of marks: ");
6          int n = sc.nextInt();
7          int[] a = new int[n];
8          System.out.println("Enter marks:");
9          for (int i = 0; i < n; i++) {
10             a[i] = sc.nextInt();
11         }
12         int shifts = 0;
13         for (int i = 1; i < n; i++) {
14             int key = a[i];
15             int j = i - 1;
16             while (j >= 0 && a[j] > key) {
17                 a[j + 1] = a[j];
18                 j--;
19                 shifts++;
20             }
21             a[j + 1] = key;
22             System.out.print("After pass " + i + ": ");
23             for (int k = 0; k < n; k++) {
24                 System.out.print(a[k] + " ");
25             }
26             System.out.println();
27         }
28         System.out.print("Final sorted list: ");
29         for (int i = 0; i < n; i++) {
30             System.out.print(a[i] + " ");
31         }
32         System.out.println();
33         System.out.println("Total shifts = " + shifts);
34     }
35 }
```

2. Insertion Sort – Marks — Run

Input used

```
5
50 20 40 10 30
```

Output

```
Enter number of marks: 5
Enter marks:
50 20 40 10 30
After pass 1: 20 50 40 10 30
After pass 2: 20 40 50 10 30
After pass 3: 10 20 40 50 30
After pass 4: 10 20 30 40 50
Final sorted list: 10 20 30 40 50
Total shifts = 7
```

Program finished successfully

3. Infix to Postfix using Stack

unit_2_a.java

```
1  import java.util.Scanner;
2  import java.util.Stack;
3  public class unit_2_a {
4      static int precedence(char ch) {
5          if (ch == '^') return 3;
6          if (ch == '*' || ch == '/') return 2;
7          if (ch == '+' || ch == '-') return 1;
8          return 0;
9      }
10     public static void main(String[] args) {
11         Scanner sc = new Scanner(System.in);
12         System.out.print("Enter infix expression: ");
13         String infix = sc.nextLine();
14         Stack<Character> stack = new Stack<>();
15         String postfix = "";
16         for (int i = 0; i < infix.length(); i++) {
17             char ch = infix.charAt(i);
18             if (Character.isLetterOrDigit(ch)) {
19                 postfix = postfix + ch;
20             } else if (ch == '(') {
21                 stack.push(ch);
22             } else if (ch == ')') {
23                 while (!stack.isEmpty() && stack.peek() != '(') {
24                     postfix = postfix + stack.pop();
25                 }
26                 stack.pop();
27             } else {
28                 while (!stack.isEmpty()
29                     && stack.peek() != '('
30                     && precedence(stack.peek()) >= precedence(ch)) {
31                     postfix = postfix + stack.pop();
32                 }
33                 stack.push(ch);
34             }
35         }
36         while (!stack.isEmpty()) {
37             postfix = postfix + stack.pop();
38         }
39         System.out.println("Postfix expression: " + postfix);
40     }
41 }
```

3. Infix to Postfix using Stack — Run

Input used

`A+B*C`

Output

```
Enter infix expression: A+B*C
Postfix expression: ABC*+
```

Program finished successfully

4. Circular Queue using Array

unit_2_b.java

```
1  import java.util.Scanner;
2  public class unit_2_b {
3      static final int SIZE = 5;
4      static int[] queue = new int[SIZE];
5      static int front = -1;
6      static int rear = -1;
7      static void insert(int value) {
8          if ((rear + 1) % SIZE == front) {
9              System.out.println("Queue Overflow");
10             return;
11         }
12         if (front == -1) {
13             front = 0;
14         }
15         rear = (rear + 1) % SIZE;
16         queue[rear] = value;
17         System.out.println(value + " inserted");
18     }
19     static void delete() {
20         if (front == -1) {
21             System.out.println("Queue Underflow");
22             return;
23         }
24         System.out.println(queue[front] + " deleted");
25         if (front == rear) {
26             front = -1;
27             rear = -1;
28         } else {
29             front = (front + 1) % SIZE;
30         }
31     }
32     static void display() {
33         if (front == -1) {
34             System.out.println("Queue is empty");
35             return;
36         }
37         System.out.print("Queue: ");
38         int i = front;
39         while (true) {
40             System.out.print(queue[i] + " ");
41             if (i == rear) break;
42             i = (i + 1) % SIZE;
43         }
44         System.out.println();
45     }
46     public static void main(String[] args) {
47         Scanner sc = new Scanner(System.in);
48         while (true) {
49             System.out.println("\n1. Insert");
50             System.out.println("2. Delete");
51             System.out.println("3. Display");
52             System.out.println("4. Exit");
53             System.out.print("Enter choice: ");
54             int choice = sc.nextInt();
55             switch (choice) {
56                 case 1:
57                     System.out.print("Enter value: ");
58                     int value = sc.nextInt();
59                     insert(value);
60                     break;
```

4. Circular Queue using Array (code continued)

unit_2_b.java

```
61         case 2:
62             delete();
63             break;
64         case 3:
65             display();
66             break;
67         case 4:
68             return;
69         default:
70             System.out.println("Invalid choice");
71     }
72 }
73 }
74 }
```

4. Circular Queue using Array — Run

Input used

```
1
10
1
20
3
2
3
4
```

Output

```
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 1
Enter value: 10
10 inserted

1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 1
Enter value: 20
20 inserted

1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 3
Queue: 10 20

1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 2
10 deleted

1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 3
Queue: 20

1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 4
```

5. Singly Linked List – Insert, Search and Delete

unit_3_a.java

```
1  import java.util.Scanner;
2  public class unit_3_a {
3      static class Node {
4          int data;
5          Node next;
6          Node(int data) {
7              this.data = data;
8              this.next = null;
9          }
10     }
11     static Node head = null;
12     static void insertBeginning(int value) {
13         Node newNode = new Node(value);
14         newNode.next = head;
15         head = newNode;
16     }
17     static void insertEnd(int value) {
18         Node newNode = new Node(value);
19         if (head == null) {
20             head = newNode;
21             return;
22         }
23         Node temp = head;
24         while (temp.next != null) {
25             temp = temp.next;
26         }
27         temp.next = newNode;
28     }
29     static void search(int value) {
30         Node temp = head;
31         while (temp != null) {
32             if (temp.data == value) {
33                 System.out.println(value + " found");
34                 return;
35             }
36             temp = temp.next;
37         }
38         System.out.println(value + " not found");
39     }
40     static void delete(int value) {
41         if (head == null) {
42             System.out.println("List is empty");
43             return;
44         }
45         if (head.data == value) {
46             head = head.next;
47             System.out.println(value + " deleted");
48             return;
49         }
50         Node temp = head;
51         while (temp.next != null && temp.next.data != value) {
52             temp = temp.next;
53         }
54         if (temp.next == null) {
55             System.out.println(value + " not found");
56         } else {
57             temp.next = temp.next.next;
58             System.out.println(value + " deleted");
59         }
60     }
```

5. Singly Linked List – Insert, Search and Delete (code continued)

unit_3_a.java

```
61     static void display() {
62         Node temp = head;
63         System.out.print("List: ");
64         while (temp != null) {
65             System.out.print(temp.data + " -> ");
66             temp = temp.next;
67         }
68         System.out.println("NULL");
69     }
70     public static void main(String[] args) {
71         Scanner sc = new Scanner(System.in);
72         insertEnd(10);
73         insertEnd(20);
74         insertBeginning(5);
75         System.out.println("After insertion:");
76         display();
77         System.out.print("Enter roll number to search: ");
78         int searchValue = sc.nextInt();
79         search(searchValue);
80         System.out.print("Enter roll number to delete: ");
81         int deleteValue = sc.nextInt();
82         delete(deleteValue);
83         System.out.println("Updated list:");
84         display();
85     }
86 }
```

5. Singly Linked List – Insert, Search and Delete — Run

Input used

```
20
10
```

Output

```
After insertion:
List: 5 -> 10 -> 20 -> NULL
Enter roll number to search: 20
20 found
Enter roll number to delete: 10
10 deleted
Updated list:
List: 5 -> 20 -> NULL
```

Program finished successfully

6. Doubly Linked List – Browser History

unit_3_b.java

```
1  public class unit_3_b {
2      static class Node {
3          String page;
4          Node prev;
5          Node next;
6          Node(String page) {
7              this.page = page;
8          }
9      }
10     static Node head = null;
11     static Node tail = null;
12     static void insert(String page) {
13         Node newNode = new Node(page);
14         if (head == null) {
15             head = tail = newNode;
16         } else {
17             tail.next = newNode;
18             newNode.prev = tail;
19             tail = newNode;
20         }
21     }
22     static void delete(String page) {
23         Node temp = head;
24         while (temp != null && !temp.page.equals(page)) {
25             temp = temp.next;
26         }
27         if (temp == null) {
28             System.out.println("Page not found");
29             return;
30         }
31         if (temp.prev != null)
32             temp.prev.next = temp.next;
33         else
34             head = temp.next;
35         if (temp.next != null)
36             temp.next.prev = temp.prev;
37         else
38             tail = temp.prev;
39         System.out.println(page + " deleted");
40     }
41     static void displayForward() {
42         Node temp = head;
43         System.out.print("First to last: ");
44         while (temp != null) {
45             System.out.print(temp.page + " ");
46             temp = temp.next;
47         }
48         System.out.println();
49     }
50     static void displayBackward() {
51         Node temp = tail;
52         System.out.print("Last to first: ");
53         while (temp != null) {
54             System.out.print(temp.page + " ");
55             temp = temp.prev;
56         }
57         System.out.println();
58     }
59     public static void main(String[] args) {
60         insert("Google");
```

6. Doubly Linked List – Browser History (code continued)

unit_3_b.java

```
61         insert("YouTube");
62         insert("GitHub");
63         insert("LeetCode");
64         displayForward();
65         displayBackward();
66         delete("YouTube");
67         displayForward();
68         displayBackward();
69     }
70 }
```

6. Doubly Linked List – Browser History — Run

Input used

(no input needed)

Output

```
First to last: Google YouTube GitHub LeetCode
Last to first: LeetCode GitHub YouTube Google
YouTube deleted
First to last: Google GitHub LeetCode
Last to first: LeetCode GitHub Google
```

Program finished successfully